



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutierrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Bryant Maximiliano Dzul Perez

Fecha:
29/Junio/2026

Introducción

El control de versiones es uno de los pilares fundamentales en el desarrollo de software moderno. Sin él, cualquier equipo que trabaje de forma colaborativa sobre un mismo proyecto estará expuesto a pérdida de información, conflictos entre archivos y una total falta de trazabilidad sobre los cambios realizados. Esta tarea analiza el caso del equipo de SmartLibrary, que experimentó de primera mano todas estas consecuencias al optar por un método informal de compartir archivos mediante una carpeta en la nube.

Análisis del Problema

1. ¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?

Fue no comenzar con un sistema de gestión de versiones, tomaron la decisión de compartir mediante carpetas en la nube, esto implica que todos los cambios están siempre en las mismas carpetas sin posibilidad de probar los cambios antes de que lleguen al sistema principal.

2. ¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?

Porque no existe un sistema que permite fusionar los cambios de más de un colaborador, no existe un proceso que permita comparar los cambios realizados por múltiples colaboradores. Esto también implica que la última persona en hacer un cambio no tenga los cambios más actuales de un desarrollador que estuviese trabajando en paralelo, es decir que, sin un sistema para fusionar, habrían pérdidas en cambios realizados por otros desarrolladores.

3. ¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?

Más desarrolladores implica más archivos y funcionalidades en desarrollo, por tanto la cantidad de conflictos es mayor y sin posibilidad de una rápida revisión para fusionar el código o escoger el necesario para el sistema principal.

Importancia del Control de Versiones

¿Qué es un sistema de control de versiones?

Permite registrar y gestionar todos los cambios realizados en un proyecto. Permite detectar quién, cuándo y por qué hizo cambios para mantener un control incluso en cuanto a seguridad. Permite que varias personas trabajen sobre el mismo proyecto de forma simultánea y permite fusionar los cambios y resolver posibles conflictos.

¿Cuál es su principal objetivo?

Garantiza un proyecto sostenible a largo plazo, trabajo colaborativo, ordenado y seguro.

¿Qué ventajas ofrece frente al manejo tradicional de archivos?

Elimina la pérdida de trabajo accidental, la colaboración de múltiples desarrolladores trabajando en paralelo, historial de cambios realizados con fecha, nombre y razón de los cambios, permite revertir los cambios en caso de romper el sistema, facilita la organización del trabajo mediante el uso de rama para funcionalidades o fixes.

Propuesta de organización

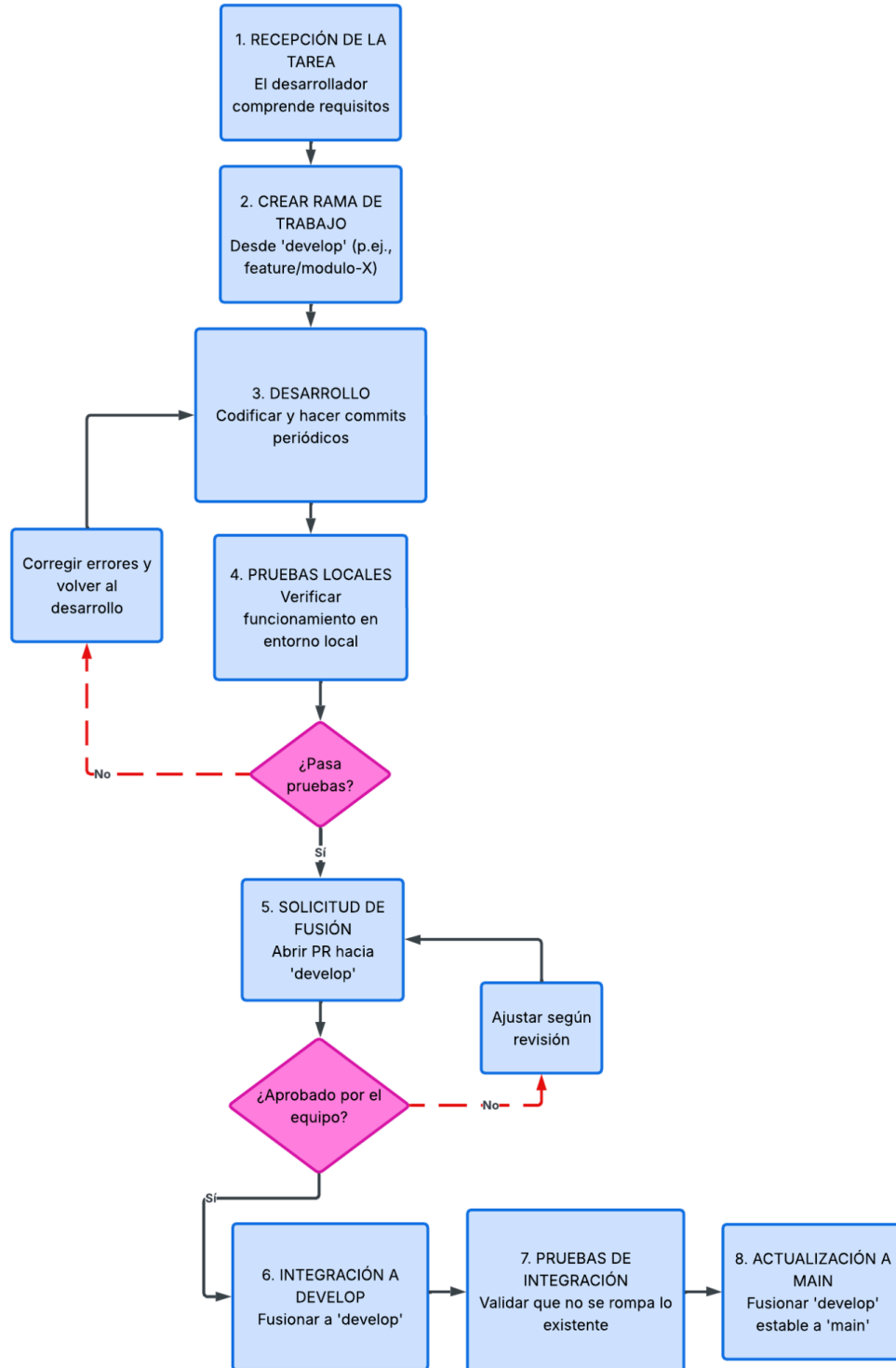
Se utilizará una rama main que sería la que contenga una versión estable del sistema, nadie desarrollaría directamente en esta rama.

Una rama develop para el merge de todos los cambios realizados por los desarrolladores, con el fin de probar los cambios en conjunto antes de enviarlos a la rama principal.

Ramas por funcionalidades grandes, como módulo de autenticación, módulo de usuarios, módulo de préstamos, módulo de reportes y una rama fix para corregir los errores que surjan en develop.

Los cambios deberían integrarse cuando un desarrollador termina su módulo y es subido a develop, así con el resto de módulos siendo integrados a develop, por medio de pull request o bien el líder del equipo realizando merge, y una vez asegurando su estabilidad se hace merge hacia main.

Flujo de Trabajo



Análisis de situaciones

1. ¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?

Git detecta si existen conflictos y notifica, entonces se debe resolver dichos conflictos decidiendo manualmente qué cambios conservar. Por lo general resuelve adecuadamente estos casos logrando fusionar por completo los cambios.

2. ¿Por qué es importante conservar un historial de cambios en un proyecto?

Permite detectar qué cambios se realizaron, quién y cuándo se realizaron. Permite leer el historial de cambios en el proyecto para entender lo que cambió y de ser necesario revertir los cambios.

3. ¿Qué beneficios aporta trabajar mediante ramas independientes?

Permite trabajar con una versión estable del sistema y de manera aislada de los demás, por tanto, si un desarrollador tiene errores al desarrollar una funcionalidad, este error solo existiría en su rama sin afectar al resto del equipo.

4. ¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?

Ayuda de tal manera que en la mayor medida de lo posible los errores se detectan en ramas aisladas antes de que lleguen a la principal, el equipo tiene la opción de revertir cambios o crear una rama para aplicar una corrección.

5. ¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?

Porque permite un mayor control en cuanto a la cantidad de cambios que se realizan en el proyecto, cada persona puede desarrollar a su ritmo y si tiene algún error lo puede reparar en su propia rama antes de integrarse con las demás en develop, al igual que permite revertir los cambios y tener un historial de estos.

Conclusión

Lo que más llama la atención del caso es que los problemas que enfrentaron no surgieron por falta de habilidad técnica de los desarrolladores, sino por la ausencia de un sistema de organización adecuado. Cinco personas talentosas trabajando sin coordinación producen caos; cinco personas trabajando con Git y una estrategia de ramas bien definida pueden construir algo robusto y escalable.

Git se ha convertido en el estándar de la industria precisamente porque resuelve de forma elegante y eficiente todos los problemas que ilustra este caso: registra cada cambio, identifica a su autor, permite el trabajo paralelo, gestiona los conflictos y garantiza que siempre exista una versión estable disponible. Además, su naturaleza distribuida significa que cada desarrollador tiene una copia completa del repositorio, lo que también actúa como respaldo automático. La enorme comunidad que lo rodea, la integración con plataformas como GitHub o GitLab y su compatibilidad con prácticamente cualquier flujo de trabajo lo consolidan como una herramienta imprescindible.